CS 6180 · FOUNDATIONS OF GENERATIVE AI · FINAL PROJECT

Microstrip Field Diffusion

Predict the electric field around a circuit-board trace from four design numbers, two ways: a regression network and a diffusion model. Then find out whether the diffusion model is worth its cost.

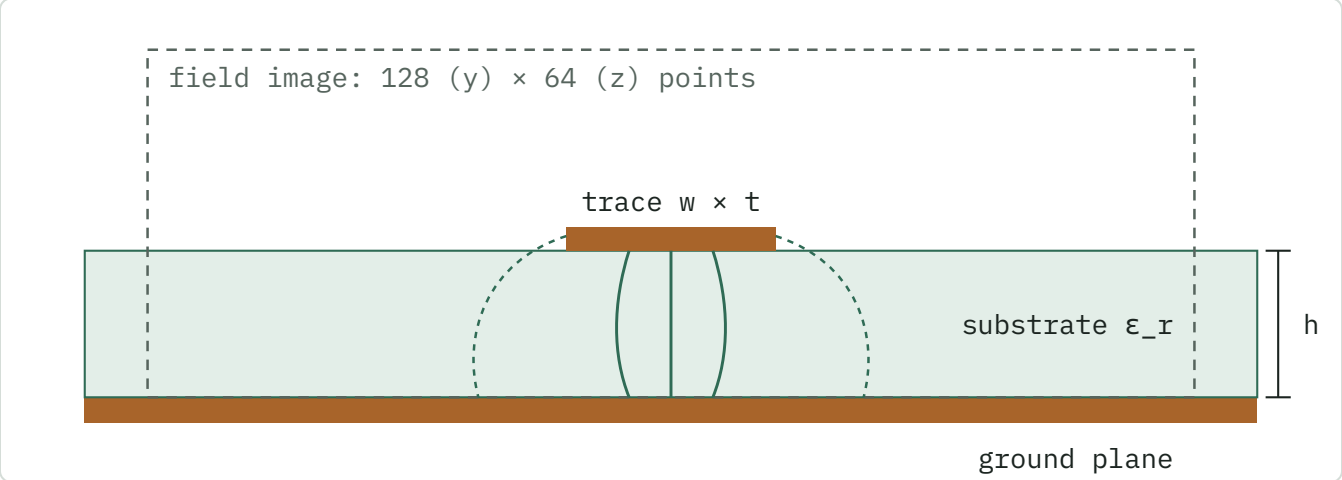
Team size: 3 · Data provided by course staff

---

1. Background
2. The data
3. The task
4. Two models
5. How to compare
6. Reading list

Background

A **microstrip** is a copper trace on an insulating board (the substrate), with a copper ground plane underneath. Its geometry is described by the substrate thickness `h`, the trace width `w` and thickness `t`, and the substrate's permittivity `ε_r`. For 9,789 designs, a solver computed the electric field in the cross-section as a two-channel image. The field is smooth in most of the image and very sharp at the trace's corners, which makes it a good test of regression versus diffusion.



Cross-section of a microstrip. The electric field (green) runs from the trace to the ground plane. The data stores it as an image over the dashed box. Not to scale.

The data

One file, `em2d_task.h5` (0.63 GB), with all preprocessing already done. N = 9,789 designs. On the cluster it is at `/courses/CS6180.202710/shared/final_proj/em2d_task.h5`.

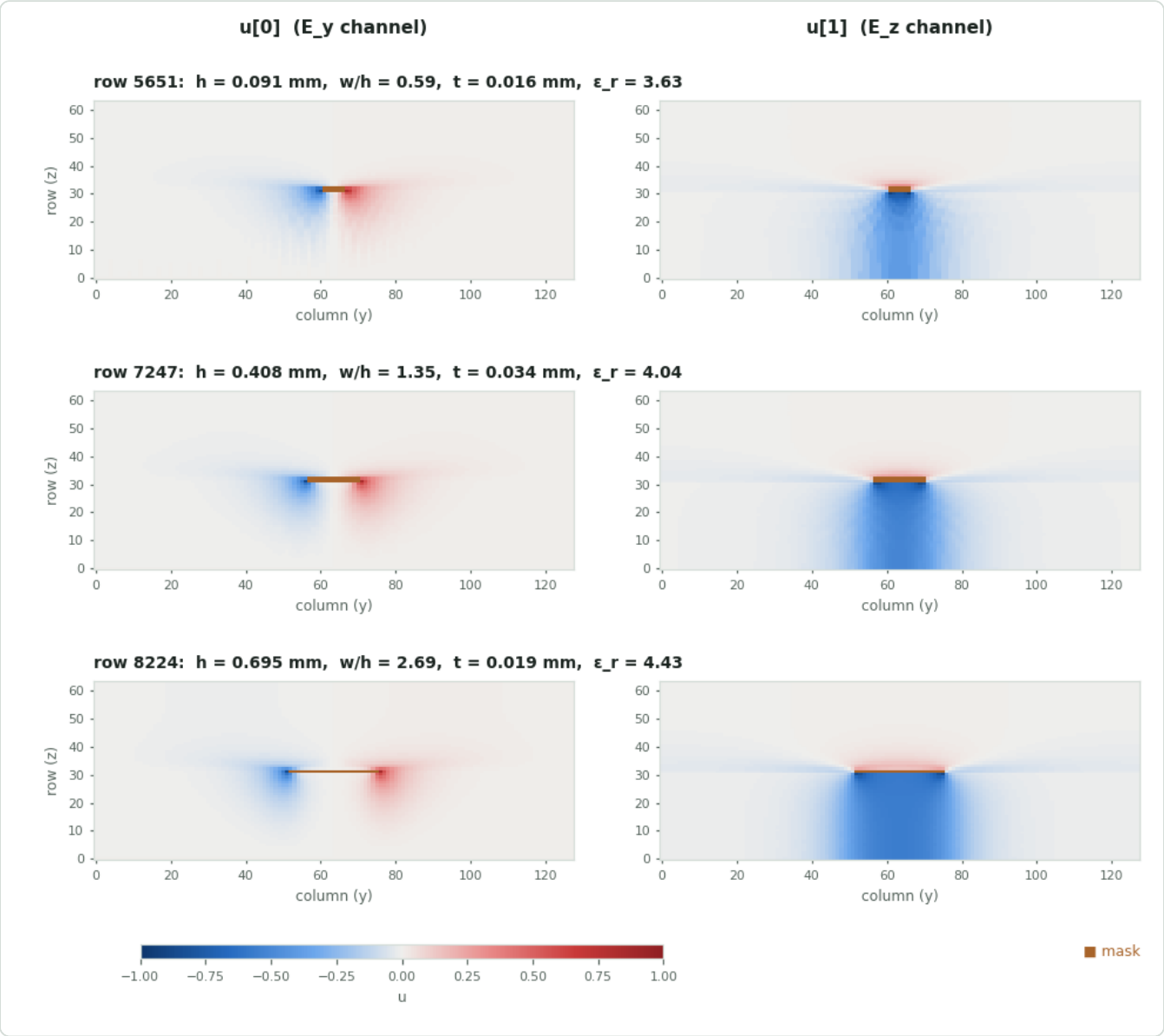
ARRAY	SHAPE	MEANING
`x`	`(N, 4)`	Input: `h`, ln(`w/h`), `t`, `ε_r`, each scaled to [−1, 1]
`u`	`(N, 2, 64, 128)`	Target image: the field divided by its peak, channels (E_y, E_z), values in [−1, 1]; rows = height z, columns = horizontal position y
`mask`	`(N, 64, 128)`	True = pixel on or next to the copper trace; ignore it
`y_over_h`, `z_over_h`	`(N, 128)`, `(N, 64)`	Pixel coordinates in units of `h`
`split/train`, `val`, `test`, `test_extrap`	`7,048 / 881 / 881 / 979`	Row indices of the fixed split
`analysis/params_raw`	`(N, 4)`	Unscaled `h` (mm), `w/h`, `t` (mm), `ε_r`, for plots only

```
import h5py

with h5py.File("/courses/CS6180.202710/shared/final_proj/em2d_task.h5", "r") as f:
    x, u, mask = f["x"][:], f["u"][:], f["mask"][:]
    yh, zh = f["y_over_h"][:], f["z_over_h"][:]
    train  = f["split/train"][:]     # also "val", "test", "test_extrap"
```

Ignore masked pixels in every loss and metric: `u` is zero there by construction, so a model that includes them is rewarded for predicting zeros next to the trace.

Example targets



`u[0]` (E_y channel) and `u[1]` (E_z channel) for three test designs: a thin substrate with a narrow trace, a medium design, a thick substrate with a wide trace. Each row is labelled with its row index in the file, and its `analysis/params_raw` values. The band at row ≈ 31 is the substrate top, where rows are bunched; copper marks the `mask`. The sharp blue and red lobes at the trace edges in `u[0]` are the detail the models must reproduce.

The task

Input	$x \in \mathbb{R}^4$
Predict	the image $u \in \mathbb{R}^{2\times64\times128}$
Loss	over unmasked pixels only
Evaluate	on u, over unmasked pixels

u is each design's field divided by its own peak value, so every image lies in [−1, 1]. The peak varies 25× across designs; predicting it is not part of the task, which is about the shape of the field.

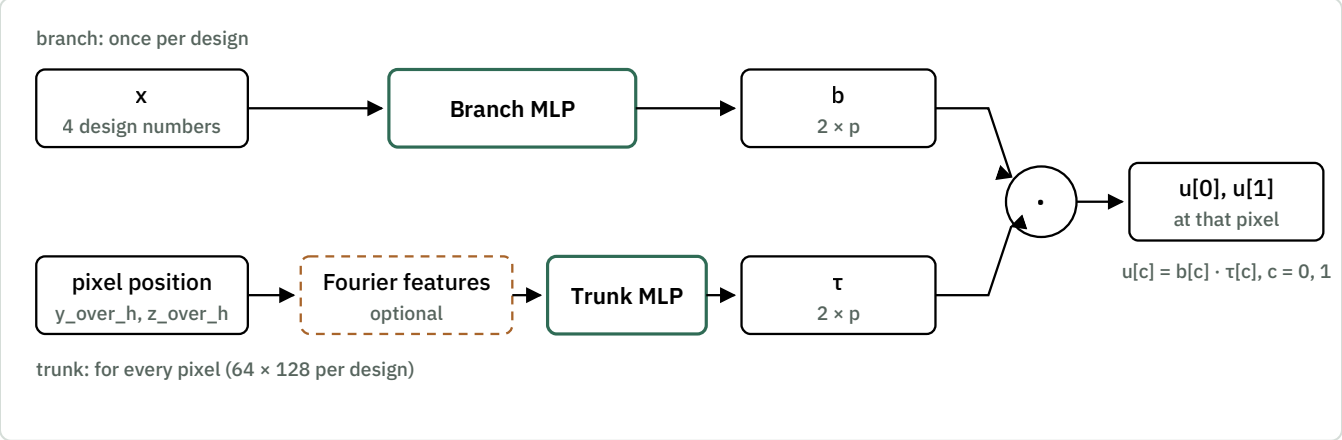
Each design has exactly one correct field, so this is regression. Networks trained with mean squared error tend to blur sharp features; diffusion models are known to keep fine detail. Whether that holds here, and at what cost, is the question. A carefully measured negative result is a good result.

Two models

> **These are starting points, not requirements**
>
> The two designs below are recommended defaults. You are welcome to propose your own variants of either one, or a model from a newer paradigm (for example flow matching, a diffusion transformer, or a neural operator). Explain your choice in the proposal, and keep the comparison fair: same data, same inputs, and a regression model alongside a generative one.

MODEL 1 · REGRESSION

DeepONet



How DeepONet predicts u. The branch turns the design into coefficients once; the trunk turns each pixel's position into basis values; for each channel, their dot product is u at that pixel. The dashed Fourier-feature step is an addition to the original design.

Branch	MLP on `x` → `b` ∈ $\mathbb{R}^{2\times p}$: one set of p coefficients per channel of `u` (for example p = 64)
Trunk	MLP on a pixel's coordinates (`y_over_h` , `z_over_h`) → `τ` ∈ $\mathbb{R}^{2\times p}$: one set of p basis values per channel. In the original DeepONet (Lu et al.) the trunk is a plain MLP on the raw coordinates. Plain MLPs on coordinates tend to blur sharp features, so a common addition, not part of the original design, is to feed Fourier features instead (Tancik et al.): sin and cos of 2π · B · (`y_over_h` , `z_over_h`), with B a fixed random 128 × 2 matrix. Start with the plain trunk, then add Fourier features and report the difference at the trace edges.
Output	`u[c]` at that pixel = `b[c]` · `τ[c]` , the dot product of the two length-p vectors, for c = 0, 1. Allowed alternative: a shared trunk, `τ` ∈ $\mathbb{R}^{p}$, with `u[c]` = `b[c]` · `τ` , so both channels use the same basis.
Training	masked MSE on `u`

MODEL 2 · DIFFUSION

Conditional diffusion (DDPM)

Generated	`u` (2 channels). Only `u` is noised and denoised.
Denoiser	small U-Net on 64 × 128 images (5–20 M parameters). Output: its estimate of the noise in `u` , 2 channels.
Condition	`x` , through a small MLP embedding added to the timestep embedding
Training	predict the added noise, MSE over unmasked pixels; about 1,000 noise steps
Sampling	DDIM, 20–50 steps; after every step, set the masked pixels of `u` back to 0

This is a standard conditional DDPM: the same setup as class-conditional image generation, with `x` in place of the class label. That works here because the pixel grid is already normalized per design:

- `z_over_h` is identical for every design: the rows always sit at the same heights in units of `h` .
- `y_over_h` depends only on w/h: the columns always span ±(w/h / 2 + 6).
- The mask always lies in rows 31–33; only its width and thickness vary, with w/h and t.

So the coordinates and the mask carry no information beyond `x`, and the network can learn the geometry from `x` alone. Giving it the geometry explicitly may still help it place the sharp edge lobes. Ways to try, as ablations:

- **Coordinate channels:** add `y_over_h` (repeated down every row) and `z_over_h` (repeated across every column) as two extra input channels, the same at every step; or their Fourier features.
- **Mask channel:** add `mask` as a 0/1 input channel, or a map of each pixel's distance to the trace.
- **Grid-free denoiser:** treat pixels as points with coordinates, for example a transformer over pixel tokens.

Start with the diffusion model generating `u` directly. Then try the **residual** version: it generates only the correction `u − u_DeepONet`, with the DeepONet prediction as an extra input channel. Regression gets the smooth bulk right and diffusion adds the detail; this usually trains faster.

How to compare

- **Splits:** tune on `val`, report on `test`. Report `test_extrap` separately: it holds the thickest 10% of substrates ($h \geq 0.722$ mm), never seen in training.
- **Baseline:** PCA on the training images (32 components) plus a Gaussian process from `x` to the coefficients. A model that does not beat it is not worth reporting. A ready-to-run script is at `/courses/CS6180.202710/shared/final_proj/pca_gp.py`; it needs numpy, scipy, h5py and scikit-learn, and takes about 25 minutes on 8 CPUs.
- **Fairness:** same data and inputs for every model; state each model's parameters and GPU-hours.

METRIC (ON `u`, UNMASKED PIXELS)	WHAT IT SHOWS
Relative L2 error	overall accuracy
Relative L2 error within 3 pixels of the mask	accuracy where the field is sharpest
Error of the spatial gradient	sharpness: blurred predictions score badly here even when L2 looks fine
Time per prediction	cost

For diffusion, also report how much several samples for the same design differ; the true answer is unique. Show a few test designs side by side: truth, each prediction, and the error map, zoomed on the trace.

Compute: the cluster's `courses-gpu` partition; each model trains in a few GPU-hours.

Reading list

- Ho, Jain, Abbeel. *Denoising Diffusion Probabilistic Models.* NeurIPS 2020.
- Song, Meng, Ermon. *Denoising Diffusion Implicit Models.* ICLR 2021. Fast sampling.

- Mardani et al. *Residual Corrective Diffusion Modeling for Km-scale Atmospheric Downscaling.* arXiv 2023. The residual version.
- Lippe et al. *PDE-Refiner: Achieving Accurate Long Rollouts with Neural PDE Solvers.* NeurIPS 2023. Why diffusion recovers detail regression loses.
- Lu et al. *Learning nonlinear operators via DeepONet.* Nature Machine Intelligence, 2021.
- Tancik et al. *Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains.* NeurIPS 2020. For the trunk.
- Ronneberger, Fischer, Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation.* MICCAI 2015.

CS 6180 final project. Data: 9,789 designs from a 2-D cross-section solver, preprocessed in em2d_task.h5.